

MedReader Agent

智能文献阅读 Agent

让每一篇论文都被真正读懂、用透

项目类型：智能文献阅读 Agent / 科研辅助工具

开发者：陈禹墨

所属机构：华中科技大学同济医学院

文档版本：v1.0

日期：2026 年

目 录

- 第一章 项目背景与意义..... 1
 - 1.1 文献阅读的当代困境..... 1
 - 1.2 现有解决方案的局限..... 2
 - 1.3 MedReader 的设计理念..... 2
 - 1.4 项目意义与社会价值..... 3
- 第二章 核心功能详解..... 4
 - 2.1 五面板布局总览..... 4
 - 2.2 PDF 原文阅读与图像定位..... 4
 - 2.3 智能解析（MinerU 驱动的结构化分块）..... 4
 - 2.4 六维度 LLM 深度分析..... 5
 - 维度一：科学问题..... 5
 - 维度二：论证思路..... 5
 - 维度三：实验方法与结果..... 6
 - 维度四：论证逻辑解析..... 6
 - 维度五：创新性..... 6
 - 维度六：局限性..... 6
 - 2.5 思维导图可视化..... 7
 - 2.6 Agent 提问对话..... 7
 - 2.7 原文段落智能导航..... 7
 - 2.8 用户系统与配额管理..... 8
- 第三章 应用场景..... 8
 - 3.1 医学生文献研读..... 8

3.2 研究生写综述与开题.....	9
3.3 PI 评估论文与组会汇报.....	9
3.4 跨学科研究者快速入门.....	9
3.5 期刊编辑与审稿人辅助.....	10
3.6 临床医生跟进最新研究.....	10
第四章 技术路线与架构.....	10
4.1 技术栈总览.....	11
4.2 系统架构概览.....	11
4.3 PDF 解析流程.....	11
4.4 LLM 调用策略.....	12
4.5 段落导航的 LLM 结构分析.....	12
4.6 思维导图自动布局.....	13
4.7 用户系统与配额.....	13
4.8 性能优化与容错.....	13
第五章 部署与运维.....	14
5.1 部署架构.....	14
5.2 环境变量配置.....	14
5.3 一键部署脚本.....	15
5.4 数据持久化与备份.....	15
5.5 常见运维命令.....	15
5.6 资源占用与扩展建议.....	16
第六章 项目路线图.....	16
6.1 短期计划（3-6 个月）	16
多 LLM 提供商深度集成.....	16
协作标注功能.....	17
私有化部署支持.....	17

移动端适配.....	17
6.2 中期计划（6-12 个月）	17
知识图谱构建.....	18
跨文献对比分析.....	18
引用网络可视化.....	18
6.3 长期愿景（1-2 年）	18
个人科研助手.....	18
文献综述自动生成.....	19
与实验记录系统集成.....	19
第七章 关于开发者.....	19
7.1 个人背景.....	19
7.2 开发动机.....	20
7.3 开发过程.....	20
7.4 致谢.....	20
7.5 联系方式与更多作品.....	21
7.6 版权与开源声明.....	21
7.7 持续迭代声明.....	21

(提示：右键点击目录 → 「更新域」可刷新页码)

第一章 项目背景与意义

1.1 文献阅读的当代困境

在生命科学与医学研究高速发展的今天，全球每年新增的生物医学文献超过两百万篇，仅 PubMed 一个数据库就已收录超过三千万篇论文。对于一名活跃在科研一线的研究生、临床医生或课题组负责人（PI）而言，每周需要阅读的文献量往往在十篇以上，而真正能够被精读、理解并内化为自己知识体系的论文凤毛麟角。文献阅读已经从单纯的「获取信息」演变为一场关于认知带宽与时间成本的隐形博弈。

更具体的痛点在于：一篇典型的 Cell Press 或 Nature 系列论文，篇幅在八千到一万五千字之间，包含五到十个图表、几十个缩写名词、复杂的实验逻辑链条和多层次的结论推导。即使母语为英语的研究者，完整精读一篇也至少需要两到三小时；对于以中文为母语的中国科研工作者，语言壁垒叠加专业术语壁垒，使得阅读时间往往翻倍。这种阅读负担直接导致两个后果：一是研究者只能选择性地精读少数几篇「必读文献」，对其他相关文献仅看摘要甚至只看标题；二是即使精读过的文献，几天后也难以回忆起核心论证细节，需要反复回看，进一步加剧时间消耗。

这种困境在跨学科研究中尤为突出。生物信息学、计算医学、合成生物学等新兴领域天然要求研究者跨越传统学科边界——一个生物医学背景的研究者需要理解机器学习方法，一个计算机背景的研究者需要理解湿实验原理。这种跨学科阅读要求更高的认知投入，传统阅读工具几乎无法提供有效支持。

1.2 现有解决方案的局限

面对文献阅读困境，研究者目前主要依赖三类工具，但每一类都有明显短板。第一类是传统 PDF 阅读器（如 Adobe Acrobat、Zotero、Mendeley 内置阅读器），它们解决了文献管理与基础标注的问题，但对论文内容的理解没有任何辅助——读者仍然需要逐字逐句阅读，遇到不熟悉的术语或方法依然要切换到浏览器检索。第二类是机器翻译工具（如 DeepL、Google Translate、彩云小译），它们解决了语言壁垒但翻译结果是平铺直叙的全文翻译，无法区分论文的论证层级，也无法把「Introduction」「Methods」「Results」「Discussion」等结构化信息凸显出来，读者面对的仍然是一篇冗长的文本。

第三类是近年兴起的 AI 摘要工具（如 ChatPDF、SciSpace、Elicit、Consensus），它们引入了大语言模型，可以基于 PDF 内容回答问题或生成摘要。然而，目前主流的 AI 文献工具大多停留在「摘要 + 问答」层面：摘要往往只有一两百字，远不足以反映一篇论文的完整论证逻辑；问答则依赖用户主动提问，用户不知道该问什么时，工具无法主动提供洞见。更重要的是，这些工具大多采用单一交互模式——要么纯对话、要么纯摘要、要么纯思维导图——缺乏一个整合的、能够覆盖「快速浏览 → 深度理解 → 知识沉淀」完整工作流的产品形态。

现有工具的共同问题在于：它们把文献当成「待翻译的文本」或「待摘要的语料」，而不是「待理解的知识结构」。一篇论文的真正价值，不在于它的字面内容，而在于它如何提出问题、如何设计实验、如何用证据支撑结论、又在何处存在局限。

1.3 MedReader 的设计理念

MedReader Agent 的核心理念是：把大语言模型从「翻译器」或「摘要器」升级为「阅读伙伴」。这个阅读伙伴不是被动等待用户提问，而是主动分析论文结构、提炼论证逻辑、可视化知识图谱，并在用户阅读过程中持续提供上下文。具体而言，MedReader 围绕三个设计原则展开：

- **原文优先** 任何 AI 生成的分析都必须能溯源到论文原文。MedReader 不允许 LLM 自由发挥，而是通过原文定位短语（quote）和段落锚点（heading）确保每一条分析结论都能被用户验证。这避免了 LLM 幻觉带来的误读风险。
- **多视角并行** 单一视角无法覆盖论文的全部价值。MedReader 同时提供 PDF 原文、智能解析、思维导图、Agent 对话、六维度分析五种视图，用户可以在不同视图间自由切换，从不同角度理解同一篇论文。
- **结构化输出** 论文的本质是结构化的论证，而不是线性的文字。MedReader 通过 LLM 把论文拆解为「科学问题 / 论证思路 / 实验方法与结果 / 论证逻辑解析 / 创新性 / 局限性」六个维度，每个维度都有专门的提示词工程（prompt engineering）保证输出深度。

这三个原则共同构成了 MedReader 与同类产品的差异化定位：不是更快的翻译、不是更短的摘要、不是更聪明的对话，而是一个真正理解论文结构、能持续辅助阅读的智能体（Agent）。

1.4 项目意义与社会价值

MedReader Agent 的意义可以从个人、机构、社会三个层面理解。对个人研究者而言，它直接降低了一篇论文的精读门槛——从平均两到三小时缩短到三十分钟左右，让研究者能够把更多时间用于思考而非阅读。对课题组与机构而言，它提供了一种可复用的阅读范式：不同成员对同一篇论文的六维度分析可以横向对比，避免「一千个读者有一千种理解」的混乱，提升组会讨论的效率和质量。

从更宏观的社会层面看，中国正在加快建设科技强国，科研人员数量已居世界前列，但人均科研产出和科研效率仍有提升空间。一款真正服务于中文科研工作者的智能文献阅读工具，能够显著降低语言与认知壁垒，让更多基层研究者、青年学生、临床医生跟上国际前沿。这不仅是工具层面的便利，更是科研基础设施层面的一种投入。

此外，MedReader 在设计上特意保留了对论文局限性的批判性分析维度。这一点尤为重要——在 AI 工具普遍倾向于「赞美论文」的当下，能够冷静指出实验设计缺陷、统计方法问题、因果推断风险的工具，反而更有助于培养研究者的批判性思维，避免盲目追随所谓「高分论文」。这是本项目作为科研辅助工具的一份社会担当。

第二章 核心功能详解

2.1 五面板布局总览

MedReader Agent 采用五面板布局，把一篇论文的阅读体验拆解为五个并行的视图，每个视图承担不同的认知任务。这种布局参考了现代 IDE（集成开发环境）的多面板设计理念——正如程序员在写代码时需要同时看到文件树、代码、终端、调试器，研究者在读论文时也需要同时看到原文、结构、图谱、对话、分析。五个面板从左到右、从上到下依次为：原文段落导航 / 全文框架 / 原文 PDF / 智能解析 / Agent 提问，底部为思维导图独立标签页。

左侧两栏（原文段落导航 + 全文框架）承担「定位」功能，让用户随时知道自己在论文的哪个位置、这一节属于什么层级。中间三栏（PDF / 智能解析 / 思维导图）通过标签页切换，承担「阅读」功能，三种视图对应三种阅读节奏：PDF 是逐字精读、智能解析是结构化浏览、思维导图是宏观把握。右侧（Agent 提问）承担

「对话」功能，用户可以基于全文随时提问。这种布局确保用户在阅读过程中无需频繁切换页面，所有关键信息都触手可及。

2.2 PDF 原文阅读与图像定位

MedReader 内置了基于 pdf.js 的高保真 PDF 阅读器，支持原文逐页渲染、文本选择、图表缩放。但它的核心价值不在于渲染本身，而在于与 MinerU 解析结果的深度联动——当用户在「智能解析」或「六维度分析」中点击某个原文定位短语时，PDF 阅读器会自动滚动到对应位置并高亮显示，让用户能够立即从 AI 分析跳回原文验证。这种「分析 → 原文」的双向跳转，是 MedReader 区别于普通 PDF 阅读器的关键。

更进一步的特性是图像定位。生物医学论文的图表（figures）往往承载了最关键的数据，但传统 PDF 阅读器中图表是「嵌在文字流里的图片」，难以单独查看。MedReader 通过 MinerU 解析提取出每张图表的独立图像文件，用户可以在「智能解析」视图中点击图表占位符，直接看到高分辨率原图，配合上下文文字说明，理解图表的实验逻辑。这对图表密集型的 Cell Press 论文尤其有价值。

2.3 智能解析（MinerU 驱动的结构化分块）

「智能解析」是 MedReader 的核心创新之一。它使用 MinerU Cloud API 对 PDF 进行深度解析，输出结构化的 Markdown + 块级 JSON，每个块都带有页面索引、边界框、文本层级（标题 / 段落 / 图表 / 公式）等元信息。这些块按照论文原始结构组织——H1 是大章节（Introduction / Results / Discussion / Methods），H2 是子小节，H3 是更细的层级——用户可以按章节快速跳转，也可以按块逐段阅读。

智能解析的优势在于把一篇「扁平的 PDF」转化为了「结构化的数据库」。每个段落都是一个可寻址的单元，AI 分析、用户提问、原文跳转都可以精确到段落级别，而不是粗粒度的「整篇论文」。这种细粒度的结构化是后续六维度分析、Agent 提问能够准确溯源的基础。

2.4 六维度 LLM 深度分析

「六维度分析」是 MedReader 最具特色的功能，也是与同类 AI 文献工具最大的差异化点。系统针对每一篇上传的论文，并行调用六次大语言模型，每次使用专门设计的提示词，分别从六个不同维度对论文进行深度剖析。六个维度并非随意划分，而是覆盖了一篇科研论文从「为什么做」到「怎么做」到「做出来了什么」到「做得好不好」的完整认知链条。

维度一：科学问题

回答「这篇论文要解决什么问题」。具体包括：核心科学问题的一句话表述、问题是如何从临床现象或前人研究空白中产生的、为什么这个问题值得研究、研究者的核心假设是什么。每个子问题都要求 1-2 段实质内容，避免空话。

维度二：论证思路

回答「整个研究是怎么组织的」。梳理研究由哪几个主要模块组成、各模块之间的逻辑关系（哪个模块的结论支撑了下一个模块的假设）、这种论证思路是否完整、与同类研究相比有什么特点。最终输出一条可视化的逻辑链条。

维度三：实验方法与结果

回答「做了什么实验，得到了什么结果」。逐条列出主要结论及支撑实验，包含实验类型（scRNA-seq / 流式细胞术 / 免疫荧光 / Western blot / 生存分析等）、关键数值（样本量 n、p 值、fold change）、方法原理与适用范围。

维度四：论证逻辑解析

回答「论证是否成立」。识别关键逻辑点（即「如果这个不成立，整篇文章结论就垮掉」的节点），用箭头图表示总逻辑链条，分析每个逻辑点的支撑强度，检查是否存在循环论证、过度推断或因果倒置的风险。这是批判性阅读的核心。

维度五：创新性

回答「新在哪里」。逐条列出核心创新点（最多三条），区分方法学创新 / 概念创新 / 应用创新 / 数据创新，并衍生 2-3 个 follow-up 研究方向。这有助于研究者判断论文的学术价值，并为自己寻找课题灵感。

维度六：局限性

回答「有什么不足」。从样本量、模型选择、实验设计、数据解读、统计方法等多方面分析不足，给出替代方案或改进路径，并评估这些不足对核心结论的影响。这是最容易被其他工具忽视、但对读者最有价值的维度。

2.5 思维导图可视化

MedReader 内置了基于 React Flow 和 Dagre 自动布局算法的思维导图功能。它会根据六维度分析结果，自动生成一棵以论文标题为根节点、六个维度为一级子节点、

各维度的子要点为二级子节点的树状图谱。每个节点都带有维度专属的配色，节点间连线使用平滑曲线（smoothstep edge），整体布局从左到右展开，符合中文阅读习惯。

思维导图的价值在于提供了一种「一眼看全文」的宏观视角。当用户面对一篇陌生论文时，先看思维导图可以在十秒内把握论文的整体骨架——研究什么问题、用什么方法、得到什么结论、有什么创新、有什么局限——然后再决定是否需要深入阅读某个维度。这种「先宏观后微观」的阅读策略，比直接从头读到尾更高效。

2.6 Agent 提问对话

右侧的 Agent 提问面板基于 DeepSeek 大语言模型，用户可以用自然语言对论文提问。与一般 AI 对话工具不同的是，MedReader 的对话是「上下文绑定」的——每次提问都会把当前论文的全文作为上下文传给模型，确保回答基于论文实际内容而非模型先验知识。同时，回答中引用的关键句子会附带原文定位短语，点击即可跳转到 PDF 对应位置。

Agent 还支持「智能追问」：当系统检测到用户的提问涉及某个具体的实验、图表或结论时，会主动建议相关的 follow-up 问题，引导用户深入挖掘。这种主动建议机制降低了用户「不知道该问什么」的认知负担，让对话自然展开。

2.7 原文段落智能导航

左侧的「原文段落导航」面板解决了传统 PDF 阅读中「找不到自己在哪一节」的问题。它不是简单地列出论文的 H1/H2/H3 标题——因为 MinerU 提取的 markdown 层级经常不可靠（例如 Cell Press 论文把「Novelty and Significance」标为 H1，把真正的 Results 子小节都嵌在它下面作为 H2）。MedReader 的创新在于：把这些原始标题列表交给 LLM 进行结构分析，让模型基于论文正文上下文判断哪些是真正的章节级标题（Introduction / Results / Methods / Discussion），哪些是子小节，哪些是期刊样板（Novelty and Significance / Data Availability / Author Contributions）。

分析后的标题树以卡片形式呈现：大章节（major）显示为带左侧彩条的卡片，子小节（children）显示为缩进列表项。期刊样板（metadata）则被隐藏，因为它们不是论文的真正结构，用户需要时可以在 PDF 中找到。这种基于原文的结构分析，让段落导航在不同期刊、不同格式的论文上都能保持一致的准确性。

2.8 用户系统与配额管理

MedReader 内置了完整的用户系统，支持注册、登录、个人配额管理。基于 NextAuth.js 实现的认证系统保证了用户数据隔离——每个用户只能看到自己上传的论文和分析历史。系统还为匿名访问留了口子，让首次使用者无需注册即可体验核心功能，降低使用门槛。

配额管理模块限制了每个用户每天的 LLM 调用次数和 PDF 解析次数，避免恶意使用导致 API 成本失控。普通用户每天可以分析三到五篇论文，对绝大多数科研工作者而言已经足够；管理员可以在后台调整配额、查看使用统计、收集用户反馈。这种设计在「免费试用」与「成本可控」之间取得了平衡。

第三章 应用场景

3.1 医学生文献研读

医学生是 MedReader 最核心的目标用户群体。本科高年级到研究生阶段的医学生，每周需要阅读大量文献以应对课程作业、文献综述、开题报告、学位论文等任务。但他们的痛点非常集中：英语阅读速度慢、专业术语不熟、对论文的实验逻辑缺乏整体把握能力、容易陷入「逐字翻译却不知道在说什么」的困境。

MedReader 为医学生提供的典型工作流是：先打开 PDF tab 浏览摘要和图表，建立初步印象；然后切换到思维导图，十秒内把握论文骨架；接着打开六维度分析，重点看「科学问题」和「实验方法与结果」两个维度，理解论文的核心论证；遇到不懂的地方，用 Agent 提问获取即时解答；最后回到 PDF，针对自己关心的段落精读整个流程下来，一篇原本需要两小时的论文，三十分钟就能完成深度阅读。

3.2 研究生写综述与开题

研究生在写文献综述或准备开题报告时，往往需要阅读三五十篇甚至上百篇相关文献。这个体量下，传统逐篇精读几乎不可能完成，多数人只能选择性地看几篇，其余仅看摘要。这直接导致综述深度不足、开题时对领域的把握片面。

MedReader 的六维度分析在这种场景下特别有价值——研究生可以快速浏览几十篇论文的「科学问题」和「创新性」维度，建立领域全景图；对关键论文再用 Agent 提问深入挖掘细节。这种「先广后深」的策略，让综述写作有了扎实的基础。

同时，每篇论文的「局限性」维度还能帮助研究生发现自己可以切入的研究空白，为开题选题提供灵感。

3.3 PI 评估论文与组会汇报

课题组负责人（PI）需要快速评估学生推荐的新论文、跟踪领域最新进展、为组会准备讨论要点。但 PI 的时间往往被行政事务、基金申请、教学任务切碎，能够分配给单篇论文的时间通常不超过十五分钟。在这种时间压力下，传统阅读方式几乎无法完成深度评估。

MedReader 为 PI 提供了「快速评估模式」：打开论文后直接看思维导图和六维度分析，十五分钟内就能判断这篇论文是否值得深入、是否值得让学生跟进、有哪些点可以在组会上讨论。特别是「论证逻辑解析」维度，能够帮助 PI 快速识别论文的潜在弱点——这种洞察往往是组会讨论的高价值话题。

3.4 跨学科研究者快速入门

生物信息学、计算医学、医学影像 AI 等领域天然要求跨学科合作。一个计算机背景的研究者想进入生物医学领域，面对的最大障碍不是编程能力，而是对生物医学论文的不熟悉——实验方法看不懂、生物学意义不理解、临床价值把握不准。反之亦然，生物医学背景的研究者面对机器学习论文，也会被损失函数、模型架构、评估指标等概念拦住。

MedReader 的「实验方法与结果」维度专门设计了通俗的方法解释——它不仅列出方法名称，还会解释方法的原理与适用范围，让跨学科读者能够快速理解。同时，Agent 提问支持任意深度的追问，遇到不懂的概念可以直接问「什么是 scRNA-seq」「为什么用 t-SNE 而不是 PCA」，获得基于论文上下文的即时解答。这种跨学科友好性是 MedReader 区别于纯生物医学工具的重要特征。

3.5 期刊编辑与审稿人辅助

期刊编辑和审稿人需要快速判断一篇投稿论文的质量、原创性、方法严谨性。他们的工作流程通常是：先看摘要和图表，决定是否送审；送审后仔细评估方法、结果、结论的可靠性。MedReader 的六维度分析与这个工作流程高度契合——「创新性」维度帮助编辑判断是否值得送审，「论证逻辑解析」和「局限性」维度帮助审稿人识别论文的潜在问题。

需要说明的是，MedReader 不能替代审稿人的专业判断，但可以作为审稿过程中的辅助工具——特别是在面对跨学科稿件时，审稿人可以用 Agent 提问快速理解不熟悉的方法部分。这种辅助能够提升审稿效率，缩短从投稿到决定的周期。

3.6 临床医生跟进最新研究

临床医生的工作节奏与基础研究者不同——他们白天忙于诊疗，能够用于阅读文献的时间往往碎片化地分布在通勤、午休、值班间隙。在这种碎片化时间下，传统阅读方式几乎无法完成一篇论文的精读。

MedReader 的多视图设计特别适合碎片化阅读：医生可以在手机或平板上打开 MedReader，先用思维导图五分钟把握论文骨架，下次有空时再看六维度分析中的「实验方法与结果」维度，再下次用 Agent 提问澄清疑问。每个视图都是独立的入口，无需从上次离开的地方继续，完美契合碎片化场景。这种「随时读一点」的模式，让临床医生能够真正跟上自己领域的最新进展，而不是被动等待指南更新。

第四章 技术路线与架构

4.1 技术栈总览

MedReader Agent 的技术栈选型遵循三个原则：一是选择成熟、社区活跃的开源技术，避免冷门框架的维护风险；二是优先支持服务端渲染（SSR）和边缘部署，保证首屏加载速度；三是与现有 AI 生态深度集成，方便后续扩展更多模型。具体技术栈如下表所示。

类别	技术选型
前端框架	Next.js 16 (App Router + Turbopack)
编程语言	TypeScript 5 (严格模式)
UI 组件库	shadcn/ui + Radix UI + Tailwind CSS 4
状态管理	React Hooks + 自定义 hook (无 Redux)
思维导图	@xyflow/react v12 + @dagrejs/dagre v3
后端运行时	Node.js 20 + Bun (production)

数据库 ORM	Prisma 6 + SQLite
认证系统	NextAuth.js v4
PDF 解析	MinerU Cloud API（主）+ pdfjs-dist（兜底）
LLM 提供商	DeepSeek（默认）+ OpenAI / 智谱 / Moonshot / Anthropic / 自定义
Markdown 渲染	react-markdown + remark-gfm
图标库	lucide-react
部署方式	Docker Compose + Node standalone

4.2 系统架构概览

MedReader 采用经典的 Next.js 全栈架构，前端与后端在同一个代码仓库中，通过 App Router 的 API Routes 暴露后端能力。整体可以划分为四层：表现层（5 面板 UI + 交互逻辑）、API 层（Next.js API Routes，处理上传、分析、对话、翻译等请求）、服务层（LLM 调用封装、MinerU 解析封装、Prisma 数据访问）、持久化层（SQLite 数据库 + 文件系统 uploads 目录）。

这种分层架构的好处是职责清晰：表现层只关心如何展示数据，API 层只关心请求路由和参数校验，服务层封装外部 API 调用和业务逻辑，持久化层负责数据存取。当某一层需要替换时（例如把 SQLite 换成 PostgreSQL，或把 DeepSeek 换成其他 LLM），改动可以局限在单层内，不会波及整体。

4.3 PDF 解析流程

PDF 解析是 MedReader 的技术核心之一，因为后续所有的 AI 分析、段落导航、思维导图都依赖结构化的解析结果。系统采用双轨策略：优先调用 MinerU Cloud API 获得最佳解析质量，失败时回退到 pdfjs-dist 在本地解析。

MinerU 是上海人工智能实验室开源的 PDF 解析服务，特别擅长处理学术论文这种结构复杂的文档——它能够识别双栏排版、提取图表为独立图像、保留标题层级、甚至解析数学公式。MinerU 返回的结果包含三部分：结构化的 Markdown 文本（保留 H1/H2/H3 层级）、块级 JSON（每个文本块带页面索引和边界框）、图像文件列

表（每张图表一个 jpg/png 文件）。这三部分被分别用于智能解析视图、原文段落定位、图表独立展示。

当 MinerU 不可用（例如 API Token 过期或网络故障）时，系统会回退到 pdfjs-dist 在 Node 端本地解析。pdfjs-dist 的解析质量不如 MinerU——它只能提取纯文本，无法识别标题层级，也无法提取图表——但能保证基本可用性，让用户至少看到论文的文字内容。这种「主备双轨」设计确保系统在网络异常或服务故障时仍然能够工作。

4.4 LLM 调用策略

MedReader 的 LLM 调用层设计得相当灵活，支持六大 LLM 提供商：DeepSeek（默认）、OpenAI、智谱 GLM、Moonshot Kimi、Anthropic Claude、自定义 OpenAI 兼容端点（如本地 Ollama、vLLM）。所有调用都通过统一的 callLLM 函数封装，传入 provider 名称、消息列表、配置参数，函数内部根据 provider 路由到对应的 API 端点和鉴权方式。

六维度分析采用了并行调用策略——六次 LLM 调用同时发起（Promise.all），而不是串行执行。这使得整个分析过程的耗时由「六次调用之和」变为「最慢一次调用」，整体响应时间从两到三分钟缩短到三十到六十秒。每次调用都使用专门设计的提示词（prompt），明确要求输出严格 JSON 格式，包含 title / summary / detail / keyPoints / quote / children 等字段。这种结构化输出让前端可以直接渲染，无需额外的解析逻辑。

另一个关键设计是「原文定位短语」（quote）字段。每次 LLM 调用都被要求在输出中包含一段 10-30 字的原文逐字片段，这段片段会被用于在前端跳转到 PDF 对应位置。这个机制确保了 LLM 的分析结论始终能溯源到论文原文，避免了「AI 编造内容」的风险。

4.5 段落导航的 LLM 结构分析

原文段落导航的难点在于：MinerU 提取的 H1/H2/H3 层级经常不可靠。例如 Cell Press 论文把「Novelty and Significance」标为 H1，把真正的 Results 子小节都嵌在它下面作为 H2；又比如某些论文直接列出子小节（如「梗死心脏中...」「SiglecF...」）却缺少「Results」总标题。如果直接按 MinerU 的层级渲染导航树，用户看到的会是一团乱麻。

MedReader 的解决方案是：把原始标题列表和论文正文前 8000 字符一起交给 LLM，让模型基于正文上下文（而不仅是标题文本）判断每个标题的真正性质。LLM 输出一个 2 级树结构：每个节点标记为「major」（真正的章节）或「metadata」（期刊样板），子小节归入对应 major 的 children 数组。如果论文缺少 Results 总标题，LLM 会合成一个；如果某 H2 实际上是主要章节，LLM 会把它提升为 major。

这种基于原文的结构分析，让段落导航在不同期刊、不同格式的论文上都能保持一致的准确性，是 MedReader 在用户体验上的一个重要细节优化。

4.6 思维导图自动布局

思维导图的技术实现基于 @xyflow/react（React Flow v12）和 @dagrejs/dagre 两个开源库。React Flow 负责节点渲染、拖拽、缩放、连线绘制等交互能力，Dagre 负责自动布局——它根据节点间的依赖关系，自动计算每个节点的 x/y 坐标，使整棵树看起来均匀、不重叠。

MedReader 把六维度分析结果转换为 React Flow 的节点和边：根节点是论文标题，六个维度作为一级子节点（每个维度有专属配色），各维度的子要点作为二级子节点。布局方向是从左到右（rankdir: LR），符合中文阅读习惯。节点连线使用平滑曲线（smoothstep edge），让视觉上更柔和。用户可以拖拽节点重新布局、缩放视图、点击节点跳转到对应内容。整个思维导图是「半自动」的——自动布局保证初始可读，手动调整满足个性化需求。

4.7 用户系统与配额

用户认证基于 NextAuth.js v4 实现，支持邮箱密码登录和注册。密码使用 bcrypt 哈希存储，session 通过 JWT 签名（NEXTAUTH_SECRET）。每个用户的论文、聊天历史、个人 LLM 配置都通过 Prisma 关联到用户 ID，保证数据隔离。系统还支持匿名访问，未登录用户可以体验核心功能，但配额更低，且数据不持久化。

配额管理通过 DailyQuota 表实现，记录每个用户每天的 LLM 调用次数和 PDF 解析次数。每次调用前先检查配额，超额则返回 429 错误并提示用户。管理员可以在后台调整配额、查看使用统计、收集用户反馈（点赞 / 点踩）。这种设计在「免费试用」与「成本可控」之间取得了平衡，也为未来引入付费套餐预留了扩展空间。

4.8 性能优化与容错

MedReader 在性能和容错方面做了多项优化。首屏加载方面，Next.js 16 的 Turbopack 编译器把启动时间从 webpack 的 8 秒缩短到 1 秒以内；standalone 输出模式让生产构建可以直接用 Bun 运行，无需 Node_modules，启动时间进一步缩短到 100 毫秒以内。响应速度方面，PDF 上传后默认显示 PDF 视图，避免用户在 MinerU 解析的 30-90 秒等待中看到空白页面；解析完成后自动切换到智能解析视图（除非用户已手动切换过 tab）。

容错方面，每个 LLM 调用都包裹在 try-catch 中，单个维度失败不会影响其他维度的输出，而是显示「该维度分析失败，请稍后重试」的占位提示。MinerU 解析失败时自动回退到 pdfjs-dist。数据库写入失败时记录日志但不阻塞响应。这种「优雅降级」的设计哲学确保系统在部分故障时仍能提供可用服务，而不是整体崩溃。

第五章 部署与运维

5.1 部署架构

MedReader 采用 Docker Compose 部署方案，整个系统封装在一个 Docker 容器内运行。容器基于 node:20-alpine 镜像，内置 Bun 运行时和 tini 进程管理器。两个宿主机目录通过 volume 挂载到容器内：./data 挂载到 /app/data（存放 SQLite 数据库），/uploads 挂载到 /app/uploads（存放用户上传的 PDF 和 MinerU 提取的图表）。这种设计确保容器重建后数据仍然持久。

容器对外暴露 3000 端口，通过宿主机的 3000 端口直接访问。如果需要 80/443 端口和 HTTPS，可以在前面加一层 Caddy 反向代理，自动签发 Let's Encrypt 证书。当前部署在阿里云 ECS（2 vCPU / 2 GiB / 40 GiB ESSD）上，公网 IP 为 47.253.133.131，通过 <http://47.253.133.131:3000> 直接访问。

5.2 环境变量配置

系统通过 .env.production 文件管理环境变量，docker-compose.yml 通过 env_file 字段加载。关键变量包括：DATABASE_URL（SQLite 数据库路径，默认 /app/data/custom.db）、UPLOADS_DIR（上传目录路径，默认 /app/uploads）、NEXTAUTH_SECRET（NextAuth 会话签名密钥，必须随机生成）、NEXTAUTH_URL（应

用公网 URL，用于 cookie 和回调）、MINERU_API_TOKEN（MinerU Cloud API 令牌）、DEEPSEEK_API_KEY（DeepSeek API 密钥）等。

为了方便首次部署，系统提供了 `.env.production.example` 模板，包含所有变量的说明和默认值。部署脚本 `deploy.sh` 会自动从模板生成 `.env.production`，随机生成 `NEXTAUTH_SECRET`，并根据服务器 IP 设置 `NEXTAUTH_URL`。用户只需要在部署后编辑 `.env.production` 填入正式的 API Key 即可。

5.3 一键部署脚本

`deploy.sh` 脚本封装了完整的部署流程，包含五个步骤：预检查（pre-flight checks）确保运行环境就绪；安装 Docker（如果尚未安装）；同步代码到 `/opt/medreader`（支持 git clone 或 rsync 两种方式）；准备 `.env.production` 并迁移现有数据；构建镜像并启动容器。脚本会在容器启动后等待健康检查通过（最多 90 秒），然后打印访问 URL。

后续更新代码时，只需在服务器上执行 `git pull && bash deploy.sh`，脚本会自动重新构建镜像并重启容器。整个过程通常在两到三分钟内完成，无需手动停止服务。这种「一键更新」的体验极大降低了运维成本。

5.4 数据持久化与备份

MedReader 的数据全部存储在两个宿主机目录中：`./data/custom.db` 是 SQLite 数据库文件，包含所有用户账号、论文元数据、聊天历史、使用统计；`./uploads/` 目录包含用户上传的 PDF 原文和 MinerU 提取的图像文件。这两个目录通过 Docker volume 挂载，容器删除后数据仍然存在。

备份策略非常简单：定期执行 `tar -czf medreader-backup-YYYYMMDD.tar.gz data/uploads/ .env.production` 即可。恢复时把备份文件解压到对应位置，重启容器即可。对于数据量较大的部署，建议每天自动备份并上传到对象存储（如阿里云 OSS），保留最近 30 天的备份。

5.5 常见运维命令

日常运维只需要掌握几个 Docker Compose 命令：`docker compose logs -f` 查看实时日志；`docker compose restart` 重启容器（修改 `.env.production` 后用）；`docker compose down` 停止容器；`docker compose up -d --build` 重新构建并启动（代码更新后

用)；`docker compose exec medreader sh` 进入容器调试。所有命令都在 `/opt/medreader` 目录下执行。

5.6 资源占用与扩展建议

在 2 vCPU / 2 GiB 的阿里云 ECS 上，MedReader 的资源占用约为：内存 300-400 MB (Next.js standalone + Bun + Prisma)、CPU 平均 5% (空闲时近 0，请求时瞬时 10-30%)、磁盘约 5 GB (镜像 + 数据库 + 上传文件)。这个占用水平让 2 GiB 服务器留有充足余量。

如果未来用户量增长到需要更高性能，建议优先升级到 4 GiB 内存，然后考虑把 SQLite 换成 PostgreSQL (修改 `DATABASE_URL` 即可，Prisma 会自动处理迁移)。如果上传文件量大，可以把 `uploads` 目录挂载到对象存储或 NAS。如果 LLM 调用量激增，可以在前面加 Redis 缓存重复请求，或引入任务队列 (如 BullMQ) 异步处理分析任务。这些扩展点都已经在架构层面预留，无需重写代码。

第六章 项目路线图

MedReader Agent 不是一次性完成的产品，而是一个持续迭代的科研工具。项目路线图按照短期、中期、长期三个时间维度规划，每个阶段都有明确的目标和交付物。这种渐进式的发展策略既能快速响应用户反馈，又能保证技术架构的稳定性。

6.1 短期计划 (3-6 个月)

多 LLM 提供商深度集成

当前系统虽然支持六大 LLM 提供商，但默认仍依赖 DeepSeek。短期计划是完善其他提供商的集成测试，特别是智谱 GLM-4.5 和 Moonshot Kimi K2，让用户可以根据论文类型选择最适合的模型。同时引入「模型路由」机制：摘要生成用快速模型 (如 GLM-4-Flash)，深度分析用强模型 (如 DeepSeek-Chat / GPT-4o)，在质量和成本之间取得最优平衡。

协作标注功能

支持课题组共享论文库和标注，让多个用户可以共同阅读同一篇论文、共享六维度分析结果、讨论关键问题。这是 MedReader 从「个人工具」向「团队工具」演进的关键一步，预计能显著提升在科研团队中的传播价值。

私有化部署支持

完善 Docker Compose 部署方案，提供 Helm Chart 支持 Kubernetes 部署，让科研机构可以在内网部署 MedReader，使用自建的 LLM 服务（如本地 Ollama、vLLM）。这对有数据安全要求的医院、研究所尤为重要。

移动端适配

针对手机和平板优化五面板布局，让临床医生、出差的研究者可以随时随地阅读文献。重点优化 PDF 渲染、思维导图缩放、Agent 对话的移动端体验。

6.2 中期计划（6-12 个月）

知识图谱构建

从单篇论文分析扩展到多篇论文的知识图谱构建——用户上传多篇相关论文后，系统自动识别论文间的引用关系、概念关联、方法相似性，生成可视化的知识网络。这能帮助研究者快速理解一个领域的全景，发现潜在的合作或选题机会。

跨文献对比分析

支持用户选择两到三篇论文进行横向对比——对比它们的科学问题、方法选择、结论异同、优劣分析。这对写综述、选课题、做决策非常有价值。技术上的挑战在于如何让 LLM 在多论文上下文中保持准确，预计需要引入检索增强生成（RAG）机制。

引用网络可视化

集成 Semantic Scholar API 或 OpenAlex API，获取论文的引用网络，可视化展示某篇论文的「上游」（它引用了谁）和「下游」（谁引用了它）。这能帮助用户快速定位某个领域的奠基性工作和最新进展。

6.3 长期愿景（1-2 年）

个人科研助手

把 MedReader 从「文献阅读工具」升级为「个人科研助手」——不仅帮助阅读论文，还主动推荐相关文献、跟踪领域进展、生成每周研究简报、辅助写作（综述、开题、论文初稿）。这需要引入用户画像、长期记忆、个性化推荐等能力，是一个更宏大的目标。

文献综述自动生成

用户指定一个主题和论文集合，系统自动生成结构化的文献综述初稿——包含背景、研究现状、方法对比、热点问题、未来方向等章节。这种「AI 辅助写作」不是替代研究者的思考，而是把研究者从机械的综述写作中解放出来，专注于更有价值的批判性分析。

与实验记录系统集成

把文献阅读与实验记录打通——研究者在阅读论文时获得的方法灵感、实验设计、参数选择可以直接记录到电子实验记录本 (ELN) 中，形成「读 → 想 → 做」的闭环。这需要与主流 ELN 系统（如 LabArchives、Benchling）的 API 集成，是一个更长期的工程。

第七章 关于开发者

7.1 个人背景

陈禹墨，现就读于华中科技大学同济医学院，研究方向聚焦生物信息学与计算医学。本科至研究生阶段的训练使他既具备生物医学的基础知识，又熟悉编程与数据分析方法，这种跨学科背景正是 MedReader 诞生的土壤。他长期关注文献阅读与科研工具的智能化，在使用各类 AI 文献工具的过程中，逐渐形成了对「理想文献阅读工具」的清晰想象，并最终决定亲手把它做出来。

7.2 开发动机

MedReader 的开发动机并非来自市场需求调研，而是来自日常科研中的真实痛点。陈禹墨在阅读大量生物信息学论文时，深切感受到现有工具的不足：翻译工具只能翻译文字，无法理解论文结构；摘要工具只能给出一两百字的概括，无法反映论证逻辑；对话工具虽然灵活，但用户不知道该问什么时无法主动提供洞见。这些痛点促使他思考：如果有一个工具能同时提供原文、结构、图谱、对话、分析五种视图，并且每个视图都能溯源到原文，那该多好。

这种思考最终凝结为 MedReader Agent 项目。它的核心理念——「让每一篇论文都被真正读懂、用透，而不仅仅是被翻译和堆叠」——正是陈禹墨作为科研工作

者对自己痛点的回应，也是对同类工具的反思。这种从真实需求出发的开发动机，让 MedReader 在功能设计上更贴近科研实际，而不是技术炫技。

7.3 开发过程

MedReader 是一个真正意义上的「独立开发者项目」——从产品构想、前端界面到后端逻辑、AI 提示词工程，均由陈禹墨一人完成。这种独立开发模式有利有弊：好处是设计统一、决策迅速、不会有团队协作的沟通成本；坏处是无法在广度和深度上同时推进，每个模块都需要自己从零学起。

在具体开发过程中，陈禹墨采用了「快速原型 + 持续迭代」的策略：先用最简陋的方式跑通整个流程（PDF 上传 → MinerU 解析 → LLM 分析 → 前端展示），然后逐个模块打磨细节。例如，最早的段落导航只是简单列出 H1/H2/H3，后来发现 MinerU 层级不可靠才引入 LLM 结构分析；最早的六维度分析是串行调用，后来为了响应速度改为并行；最早的思维导图节点样式混乱，经过多轮用户反馈才调整为现在的统一矩形。这种「先粗糙后精细」的迭代方式，让项目在有限时间内达到了可用的状态。

7.4 致谢

本项目受到华中科技大学同济医学院基础医学院的资助支持，在此致以诚挚感谢。学院提供的科研环境和资源支持，是本项目得以顺利推进的重要保障。同时也感谢在开发过程中给予反馈和建议的同学们，你们的真实使用场景是产品迭代最宝贵的输入。

7.5 联系方式与更多作品

陈禹墨通过公众号「行止集 BioRhythm / 计算医学」分享生物信息学、计算医学、科研工具开发等方面的思考与作品。MedReader Agent 是其在科研工具方向的探索之一，未来还会有更多围绕科研效率提升的项目陆续推出。

如有建议、合作意向或 bug 反馈，欢迎通过公众号留言联系。每一份反馈都会被认真对待，并可能影响项目的下一版迭代方向。MedReader 的发展离不开用户社区的参与，期待与各位科研工作者共同打造更好的文献阅读体验。

7.6 版权与开源声明

MedReader Agent 当前为闭源项目，所有权利归开发者陈禹墨所有。未经授权，不得将本项目用于商业用途。但开发者欢迎学术研究、教学使用等非商业场景的应用，并愿意为这类使用提供必要的支持。

项目当前处于持续迭代中，会根据用户反馈和技术发展不断优化。未来可能考虑部分模块开源（如 LLM 调用封装、PDF 解析逻辑），以回馈开源社区——本项目本身也受益于 Next.js、Prisma、MinerU、React Flow 等众多开源项目，没有它们就没有 MedReader。这种开源精神的传承，是科研工具生态健康发展的基础。

7.7 持续迭代声明

文献阅读助手 · 持续迭代中。这行小字会一直保留在 MedReader 的页脚，提醒每一位用户：你今天看到的版本不会是最终版本，明天的 MedReader 会更好。这种持续改进的承诺，是开发者对用户、对自己的一个约定——也是科研工具这种「永远不会完美」的产品最诚实的姿态。